Andas: A Columnar, Streaming Data Processing Library for Android

Technical Report on CSV Ingestion, Computation, and Performance Evaluation

Andas Engineering Team

November 1, 2026

Abstract

Andas is an Android library that brings a pandas-like data-analysis API to Kotlin and Java. The previous implementation materialized an entire CSV file as a list of strings, constructed a map per row, and then transposed the rows into columns; each column also kept a private copy of the index and a label-to-position hash map. On a 192 MB Android heap this design ran out of memory at 100k rows × 15 columns. We redesigned the ingestion and computation paths around streaming parsing, columnar primitive storage, a shared range index, and algorithmically improved operators. We evaluate the optimized library on a JVM and on an Android ARM64 emulator, and compare it with pandas 2.1.0 on the same machine and datasets. On the JVM, the optimized reader completes 200k rows × 15 columns (36 MB) in 1.44 s with 56 MB of retained heap, whereas the baseline fails with an out-of-memory error at the same size. On the Android emulator, the baseline fails at 100k rows and the optimized reader completes 200k rows with 35 MB of retained heap. At 50k rows, statistical, sorting, and join operations improve by 7–33× over the baseline; filtering and grouped aggregation improve by 2.9× and 1.4×, respectively. Compared with pandas, the optimized reader is still 3.2–6.0× slower on the desktop JVM but uses less memory, while computation kernels remain 6–20× slower, pointing to native columnar kernels as future work. A 35-metric cross-language consistency check (pandas vs JVM vs Android) passes on both runtimes with a maximum relative error of 3.17×10^{-15}.

Keywords: Android, CSV parsing, columnar storage, JVM performance, pandas, data analysis.

1 Introduction

Data analysis on Android is usually performed either on a remote service or with ad-hoc collections and loops. Andas explores a third option: a local, pandas-like library for Kotlin and Java, implemented as a small JVM API with optional JNI acceleration. The library supports `Series`, `DataFrame`, selection, filtering, sorting, grouping, joins, null handling, asynchronous I/O, and CSV import/export.

The first release prioritized API coverage. Its CSV reader followed a conventional but memory-hungry pipeline: the file was read into a `List<String>`, each line was parsed into a `Map<String, Any?>`, and the list of maps was transposed into column-oriented `Series` objects. This pipeline is acceptable for small examples but not for the workloads that motivate a data library. In our measurements, a 36 MB file with 200k rows and 15 columns could not be read on a 192 MB Android heap, and even on a 256 MB JVM heap it failed at 200k rows. The same design also resulted in slow statistical kernels, quadratic sorting, and quadratic joins.

This report describes the redesign, measures it, and compares it with pandas as a reference implementation. The contributions are:

- a streaming, column-oriented CSV parser that never materializes the whole file or row maps;

- primitive column storage (`IntArray`, `LongArray`, `DoubleArray`, `BooleanArray`) with a nullable bitmap, plus a shared $O(1)$-memory range index;
- algorithmic rewrites of statistics, sorting, filtering, grouping, and joins;
- an evaluation on the JVM and Android, including memory, out-of-memory thresholds, and a pandas comparison.

The implementation is available at `https://github.com/trueWangSyutung/Andas` and is released as version 0.1.5. The raw results, scripts, and figures are included in the repository under `bench/`.

2 Background and Motivation

2.1 Android memory constraints

Android applications run on ART with a per-application heap limit. The limit depends on the device and on whether `android:largeHeap` is requested; in our API-36 ARM64 emulator, `Runtime.maxMemory()` reports 192 MB. A data library must therefore keep the live set small and avoid transient object graphs that are several times larger than the input. The legacy pipeline violated both requirements: it created one map entry per cell during parsing, retained the entire row representation, and then created a second column representation.

2.2 Legacy pipeline

For n rows and m columns, the legacy reader performed the following steps:

1. read the whole file into `List<String>` (one string per line);
2. for each line, split fields into a `List<String>`, then build a `Map<String, Any?>` with m entries;
3. for each column, scan all row maps and build a `Series`, copying the data list;
4. create a default index list of n boxed integers for every column and a `HashMap` from label to position for every column.

The row maps, boxed numeric values, duplicated indices, and transposition copy dominate both time and memory. In addition, type inference used only the first data row, so a leading empty value could turn a numeric column into a string column.

3 Design and Implementation

3.1 Streaming, columnar CSV ingestion

The new reader is implemented in `CsvReader`. It reads characters through a 64 KB buffer and feeds a small state machine that understands quoted fields, escaped quotes (`""`), embedded commas and newlines, CRLF, a UTF-8 BOM, and blank lines. Fields are appended to reusable buffers; the parser does not allocate a string for every numeric cell.

Column types are inferred from up to 64 sample records. The inference merges the observed kinds (int, long, double, boolean, date, string) and selects the narrowest common type. If a later value cannot be parsed according to the inferred type, the column is promoted to string storage so that no data are lost. Duplicate headers are disambiguated (`a`, `a` becomes `a`, `a_1`).

Figure 1 shows the ingestion pipeline. A column builder appends directly into primitive arrays; a null bitmap is allocated only when the first null is observed. At the end, the builders produce `Series` objects that share a single index instance.

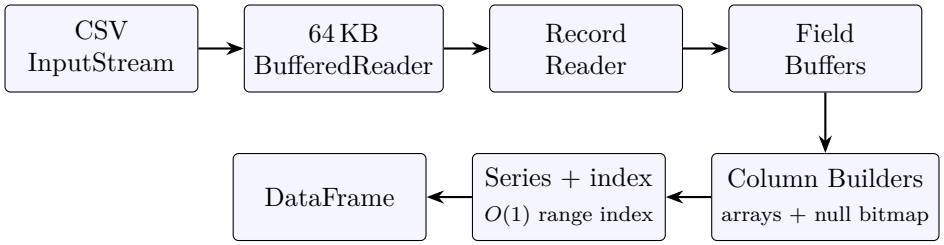


Figure 1: Streaming, columnar CSV ingestion. No row maps or full-file string list are created.

The core loop is intentionally simple; all complexity is hidden in the record reader and the column builders.

```
while (reader.nextRecord()) {
  for (int c = 0; c < columnCount; ++c) {
    if (c < reader.fieldCount())
      builders[c].append(reader.field(c));
    else
      builders[c].appendNull();
  }
}
return DataFrame::fromColumnar(builders, RangeIndex::of(rowCount));
```

Batched ingestion uses the same parser with a row-count trigger: every batch is converted into an independent `DataFrame` and handed to a callback, so peak memory is bounded by the batch size rather than by the file size.

3.2 Shared index and lazy label mapping

Every column of a `DataFrame` shares the same index object. For the common default index $0, \ldots, n-1$, `RangeIndexList` stores only the size and computes labels on demand, taking $O(1)$ memory instead of $O(n)$ boxed integers. Custom string or integer labels are stored once and shared by all columns. The label-to-position `HashMap` is built lazily, only when a label lookup is actually requested; positional access, which dominates computation, never needs it.

3.3 Primitive column storage

Numeric and boolean columns are stored in primitive arrays. When a column contains nulls, a `BooleanArray` bitmap marks present positions; otherwise the bitmap is omitted. The public `List<T?>` view is a lightweight adapter that boxes values only when a caller explicitly reads them through the `List` interface. Internal kernels use the arrays directly, avoiding both boxing and intermediate lists.

3.4 Algorithmic rewrites

Table 1 summarizes the changes. The common theme is to remove intermediate collections and to replace quadratic scans with sorting or hashing.

Table 1: Legacy versus optimized implementation.

Operation	Legacy	Optimized
Read	whole file as strings, row maps, transpose	streaming records into primitive columns
Index	per-column list and hash map	shared range index, lazy map
Statistics	`filterNotNull` → `map` → `toDoubleArray`	single pass over primitive arrays
Sort	string comparison, `indexOf` per row ($O(n^2)$)	numeric sort ($O(n \log n)$), stable row permutation
Filter	full row maps and per-cell `Series`	lazy row view, position mask
Group-by	per-group temporary `DataFrame`	one hash grouping and one aggregation scan
Join	nested loops ($O(nm)$)	hash join ($O(n+m)$)
Export	unbuffered, unescaped writes	buffered writer with CSV escaping

The statistical kernels traverse a column once. Variance and standard deviation use Welford's online algorithm, which avoids the cancellation error of the naive sum-of-squares formula and allows the mean and variance to be computed in the same pass. Sorting compares numeric values numerically (the legacy Kotlin fallback compared their string representations), places nulls last, and builds the output columns by permutation rather than by repeated label lookups. Filtering collects positions and projects the columns once. Grouped aggregation hashes the group key once and aggregates all requested columns in a second pass. Joins build a hash table on one side and probe it with the other.

3.5 Native layer and OpenMP

The optional JNI layer exposes array kernels for sums, means, extrema, normalization, dot products, norms, sorting indices, null handling, and grouping. We enabled OpenMP in the CMake build via `find_package(OpenMP)`; if OpenMP is unavailable, the build falls back to serial code. We also fixed index-order bugs in the parallel implementations of `findNullIndices`, `dropNullValues`, `where`, and `mergeIndices`: merging per-thread vectors under a critical section does not preserve index order, which silently misaligned rows.

3.6 Compatibility and correctness

The public API is preserved: existing constructors, `readCSV`, `sum`, `sortValues`, `groupBy`, `merge`, and asynchronous helpers keep their signatures. The implementation adds regression tests for quoted fields, embedded newlines, duplicate headers, missing columns, type promotion, null handling, numeric sorting, grouped aggregation, joins, correlation, and normalization. On the JVM, 32 tests pass and 9 JNI-only tests are skipped because the native library is not available; the Android instrumentation test exercises the reader on the emulator.

4 Experimental Methodology

4.1 Environment

Experiments run on an Apple M2 machine with macOS 27.0.1, OpenJDK 21.0.12, Gradle 8.13, AGP 8.13.2, and Kotlin 2.0.21. Android measurements use an API-36 ARM64 emulator (AVD `MullionFold76`) whose reported maximum heap is 192 MB. The pandas reference runs on CPython 3.10.11 with pandas 2.1.0 and the NumPy C engine.

4.2 Datasets

We generated deterministic synthetic CSV files whose schema mirrors the project's 37 MB `largest.csv`: three 10-character string columns (the third has 10 categories), three 12-digit integer columns, three small integer columns, three double columns, and three float columns. Row counts are 10k, 25k, 50k, 100k, and 200k; the largest file is approximately 36 MB. A separate join benchmark uses two 5k-row tables with an integer key. The generation script fixes the random seed, so all implementations read identical bytes. All files are on local disk and the page cache is warm before each measured run.

4.3 Baseline and variants

The baseline is commit `9bb6715`, the last revision before this work, checked out in a separate Git worktree. On the JVM the native library is unavailable; we wrap the baseline's `System.loadLibrary` calls in a try/catch so that the baseline falls back to its Kotlin paths instead of aborting during class initialization. This patch is used only for JVM experiments. On Android the baseline is built unmodified and uses its JNI layer.

4.4 Metrics and protocol

- **Read time**: wall-clock time of `DataFrameIO.readCSV(File)`. Each size runs in three independent JVMs (or Python processes) and we report the median. A 10k-row file warms the parser and the JIT before the measured read.
- **Retained heap**: `totalMemory - freeMemory` after `System.gc()` and a 120 ms pause. For pandas we report the incremental resident set size (RSS) after the read; the two metrics are related but not identical.
- **Peak heap**: a sampler thread polls `MemoryMXBean` every 50 ms during the measured read and records the maximum. This is an upper bound on heap usage during the read, not a retained-set measurement.
- **Operation time**: after reading 50k rows, a warm-up operation is executed, then a second operation is timed; three independent JVMs provide the median.
- **Out of memory**: a run is marked OOM when an `OutOfMemoryError` is thrown. JVM reads use `-Xmx256m`; Android uses the emulator's 192 MB heap.

5 Results

5.1 JVM CSV read scalability

Table 2 and Figure 2 report read time, retained heap, and sampled peak heap. The baseline grows faster than linearly in time and memory; the optimized reader remains roughly linear and still completes at 200k rows.

Table 2: JVM CSV read performance (36 MB at 200k rows, 15 columns, `-Xmx256m`).

Rows	Time (ms)			Retained (MB)		Peak (MB)	
	Base	Opt.	Speedup	Base	Opt.	Base	Opt.
10k	321	138	2.33×	15	4	88	41
25k	623	244	2.55×	36	8	137	119
50k	1071	541	1.98×	78	14	165	141
100k	2540	964	2.63×	155	30	245	162
200k	OOM	1440	–	OOM	56	OOM	214

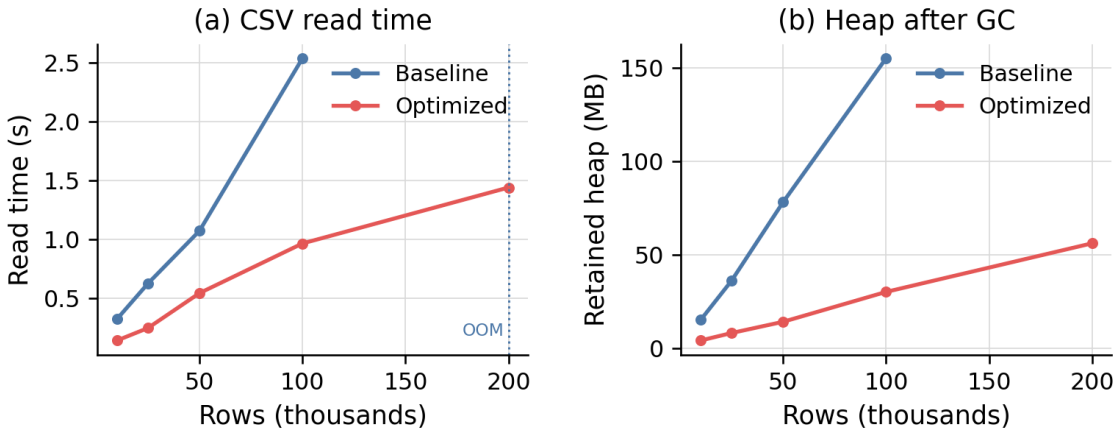


Figure 2: JVM CSV read scaling: (a) read time; (b) retained heap after GC. The baseline fails at 200k rows under a 256 MB heap.

The optimized reader uses 3–5× less retained heap at the same row count. At 100k rows the retained heap falls from 155 MB to 30 MB, and the baseline's 200k run cannot complete. The optimized peak at 200k (214 MB) is close to the heap limit, but the retained set after GC is only 56 MB; the difference is allocation churn from string cells and array growth, not live data.

5.2 Computation kernels

Table 3 and Figure 3 report operation time at 50k rows. Statistical kernels improve by 7–30×, sorting by 32.9×, and the 5k × 5k join by 30.9×. Filtering improves by 2.9× because the legacy implementation still performs a linear scan and the optimized path adds column projection. Grouped sum improves by only 1.4× because the legacy fallback was already a hash aggregation; this is expected and shows that the gains come from data structures and algorithms rather than from uniformly replacing code with JNI.

Table 3: Operation time at 50k rows (merge: 5k × 5k).

Operation	Baseline (ms)	Optimized (ms)	Speedup
`sum`	30	1	30.0×
`mean`	28	2	14.0×
`std`	28	4	7.0×
`describe`	80	3	26.7×
`sortValues`	2629	80	32.9×
`filterGreaterThan`	23	8	2.9×
`groupBySum`	27	20	1.4×
`merge`	557	18	30.9×

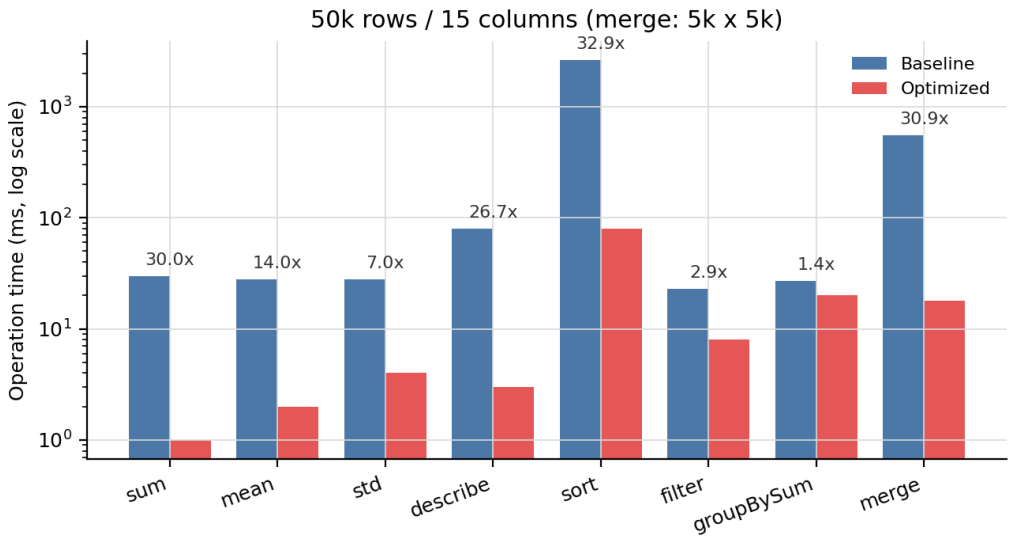


Figure 3: Operation time for the baseline and the optimized implementation (log scale).

A correctness check confirms the sorting change: the legacy Kotlin fallback sorts `double_col_10` lexicographically and returns −0.0166 as the first value on the 50k dataset, while the optimized implementation returns −99.9979 and produces a monotonically non-decreasing numeric sequence. The legacy native sort also drops nulls before computing the permutation, which misaligns rows when the sort column contains nulls.

5.3 Comparison with pandas

pandas is the direct design reference for Andas. We ran pandas 2.1.0 on the same machine and the same CSV files, using `pd.read_csv(..., engine="c")`. Each measurement uses a fresh Python process; a 10k read warms the C parser, then the target file is read. Table 4 and Figure 4 report read time; Table 5 and Figure 5 report operations.

Table 4: Read comparison with pandas 2.1.0 (same machine and files).

Rows	Baseline (ms)	Optimized (ms)	pandas (ms)	Opt./pandas	Base/pandas
10k	321	138	22.9	6.02× slower	14.0× slower
25k	623	244	56.4	4.32× slower	11.0× slower
50k	1071	541	112.8	4.80× slower	9.50× slower
100k	2540	964	239.6	4.02× slower	10.6× slower
200k	OOM	1440	446.7	3.22× slower	OOM

Table 5: Operation comparison with pandas 2.1.0 at 50k rows (merge: 5k × 5k).

Operation	Baseline (ms)	Optimized (ms)	pandas (ms)	Opt./pandas
`sum`	30	1	0.12	8.3× slower
`mean`	28	2	0.10	20.0× slower
`std`	28	4	0.37	10.8× slower
`describe`	80	3	3.22	0.93× (slightly faster)
`sortValues`	2629	80	12.67	6.3× slower
`filter`	23	8	0.55	14.5× slower
`groupBy`	27	20	1.85	10.8× slower
`merge`	557	18	1.43	12.6× slower

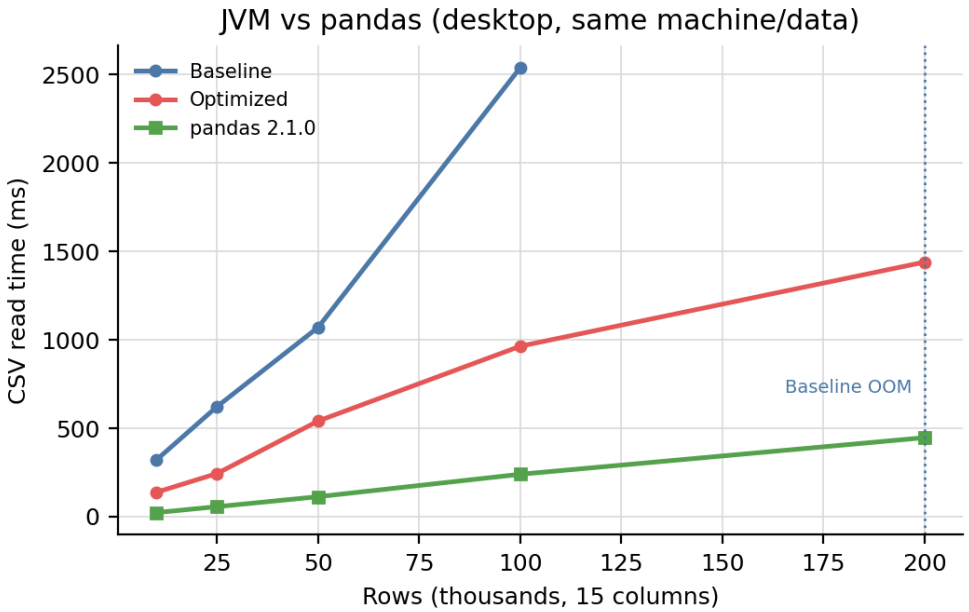


Figure 4: Read-time comparison with pandas 2.1.0. pandas uses a C parser and NumPy arrays; the runtimes differ, so the comparison indicates order of magnitude only.

Table 6: Representative consistency metrics (pandas 2.1.0 vs JVM/Android).

Metric	pandas	JVM	Android
`sum(d10)`	-7465.802624422415	-7465.802624422438	-7465.802624422438
`std(d10)`	57.606263335764076	57.60626333576399	57.60626333576399
sort first value	-99.94711091909323	-99.94711091909323	-99.94711091909323
filter count	222	222	222
`groupby cat_3`	-2092.5884491227703	-2092.588449122769	-2092.588449122769
merge rows	5000	5000	5000
`corr(d10,d11)`	0.008147698079084498	0.008147698079084522	0.008147698079084522

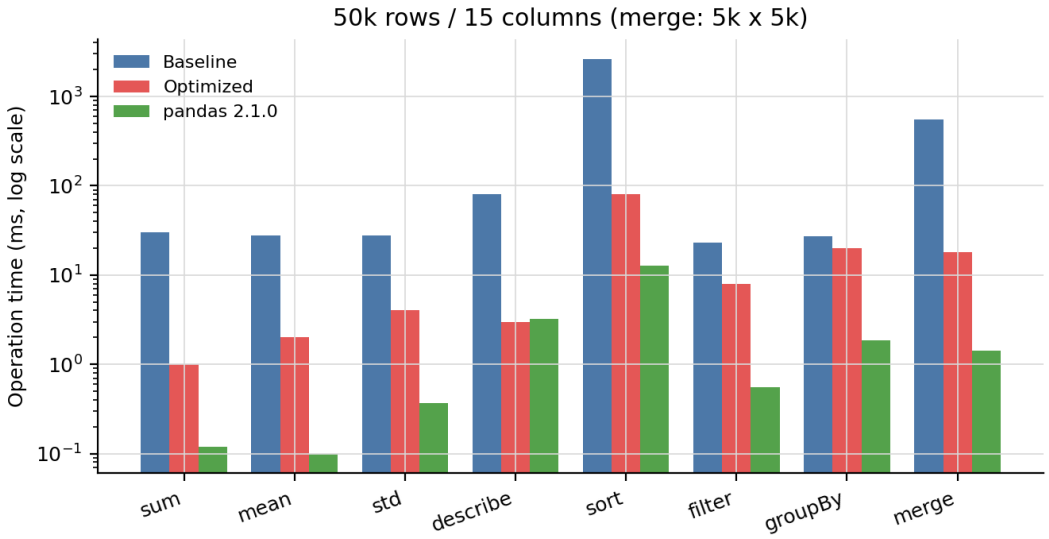


Figure 5: Operation-time comparison with pandas 2.1.0 (log scale).

pandas remains 3.2–6.0× faster at reading. Its memory increment, however, is higher than the optimized retained heap: 106.8 MB versus 56 MB at 200k, 54.2 MB versus 14 MB at 50k. The comparison is not apples-to-apples: pandas runs in CPython with a C CSV parser and contiguous NumPy arrays, while Andas runs on the JVM (and ART on Android), and RSS is not the same as a GC-retained heap. Still, it gives a useful upper bound. The optimized implementation closes most of the read-time gap of the baseline (roughly 10–14× slower than pandas) to 3.2–6.0×, and it uses less memory. Computation kernels remain 6–20× slower than pandas, except `describe`, which is slightly faster because pandas constructs several statistics and wraps them in a Series.

5.4 Cross-language consistency

Performance is necessary but not sufficient: the same input must produce the same answer in pandas, on the JVM, and on Android. We extracted the first 5000 rows of the 50k dataset and generated reference metrics with pandas 2.1.0. The JVM harness and the Android instrumentation test read the identical CSV (the Android copy is packaged as a test asset) and compute the same 35 metrics: cardinalities, ‘sum‘/‘mean‘/‘std‘/‘min‘/‘max‘/‘describe‘, sort endpoints, filter and group-by sums, join cardinality and amount, Pearson correlation, and normalized endpoints. Integer metrics must match exactly; floating-point metrics use a relative tolerance of 10^{-10}.

All 35 metrics pass on both the JVM and Android. The maximum relative error is 3.17×10^{-15}, well below the 10^{-10} threshold; integer counts and sort endpoints match exactly. The residual difference is explained by floating-point summation order (NumPy pairwise summation versus sequential accumulation), not by a semantic difference. The full table is provided in `bench/`

`results/consistency_summary.csv`.

5.5 Android emulator

Table 7 and Figure 6 report the Android experiment. The CSV is generated on the device before the timed read; only the read is measured.

Table 7: Android emulator CSV read (API 36 ARM64, 192 MB heap).

Rows	Baseline (ms)	Optimized (ms)	Baseline retained (MB)	Optimized retained (MB)
10k	1787	1192	10	1
25k	3298	2025	28	4
50k	6431	3468	56	8
100k	OOM (18.1 s)	7317	–	18
200k	OOM (64.5 s)	16042	–	35

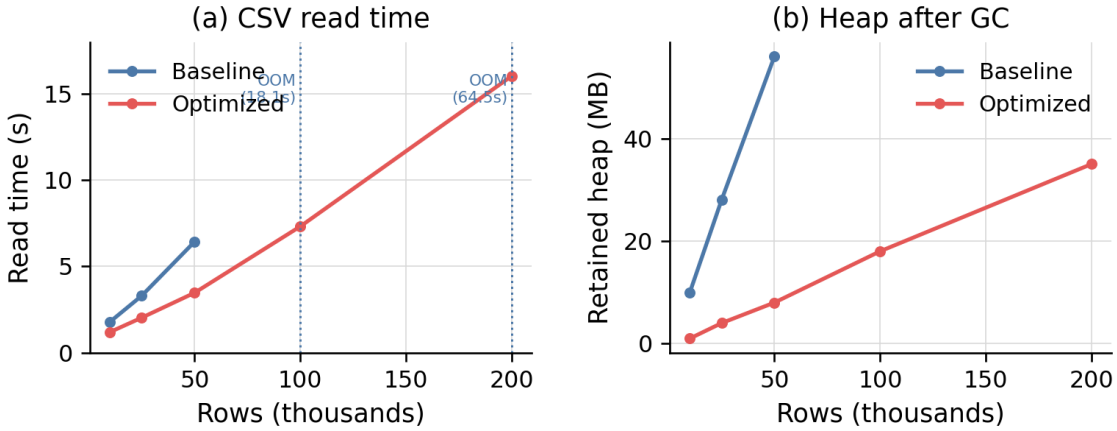


Figure 6: Android emulator results: (a) read time; (b) retained heap after GC. The baseline OOMs at 100k and 200k rows; the optimized reader completes 200k rows with 35 MB of retained heap.

The Android results reproduce the JVM trend and directly address the original failure. The legacy reader exhausts the 192 MB heap at 100k rows after 18.1 s and at 200k rows after 64.5 s. The optimized reader completes 100k rows in 7.3 s with 18 MB retained and 200k rows in 16.0 s with 35 MB retained. For the sizes that both complete, the read time falls by 33%, 39%, and 46% at 10k, 25k, and 50k rows; retained memory falls by roughly 7× at 50k.

6 Discussion

6.1 Memory complexity

For n rows, m columns, and average string length L, the legacy design retains $O(nm)$ map entries and boxed values, a transposed copy of the data, and $O(nm)$ index entries and hash-map entries. The optimized design retains $O(n)$ primitive values per numeric column, $O(nL)$ characters for string columns, and one $O(1)$ or $O(n)$ index shared by all columns ($O(1)$ for the default range index). This explains both the lower retained heap and the higher row count that fits in a fixed heap.

6.2 Where the speedups come from

The gains are not uniformly due to JNI. They come from:

- removing intermediate lists and boxing in statistical kernels (7–30×);
- changing sorting from quadratic label lookups to an $O(n \log n)$ permutation (32.9×, with the gap growing with n);
- changing joins from nested loops to a hash join (30.9×);
- streaming ingestion that avoids a full transposition (2–2.6×).

Grouped sum improves only 1.4× because the legacy fallback already used a hash map. This heterogeneity is useful: it shows which operations were actually pathological and which were already near their algorithmic optimum.

6.3 Remaining gap and future work

The comparison with pandas suggests three directions. First, a native CSV tokenizer that parses numbers directly from bytes and writes into off-heap arrays could reduce the remaining 3–6× read gap. Second, vectorized native kernels over primitive columns could remove JNI call and boxing overhead for statistics, filtering, and grouping. Third, a zero-copy column format (for example, an Arrow-compatible buffer) would allow interop with other tools and avoid materializing the `List` view. On Android, ART's JIT and GC behavior also deserve a dedicated study on physical devices, since our measurements use an emulator.

7 Threats to Validity

1. **JVM versus ART.** JVM measurements do not fully represent ART's JIT and GC behavior. We mitigate this with an Android emulator experiment, but an emulator is still not a phone.
2. **Baseline JVM fallback.** The JVM has no native library, so the baseline is patched to fall back to Kotlin. This affects absolute times for kernels that would otherwise use JNI. The Android read experiment uses the unmodified baseline, so the read conclusions are unaffected.
3. **Synthetic data.** The data distribution and string lengths differ from production workloads. The generator is seeded and deterministic, and the schema mirrors the project's real 37 MB file; we also exercise that file in unit tests.
4. **Peak sampling.** A 50 ms sampler can miss short peaks, and `System.gc()` does not guarantee immediate collection. Retained values can fluctuate by a few megabytes.
5. **pandas runtime differences.** pandas runs on CPython with C/NumPy, while Andas runs on the JVM/ART. Memory is reported as RSS increment for pandas and retained heap for Andas. The comparison is intended as an order-of-magnitude reference, not a claim that one library is universally faster.
6. **Single-machine study.** Each configuration is repeated three times and we report medians, but we do not repeat the study across machines or device models.

8 Related Work

pandas established the `DataFrame` abstraction for Python and remains the reference API for tabular data analysis [1]. Columnar in-memory formats such as Apache Arrow [5] motivate contiguous primitive buffers and zero-copy interchange; Andas adopts the columnar idea but keeps a Kotlin/JVM API. On Android, most data-processing libraries either wrap SQLite or operate on row-oriented collections; Andas targets analytic operations over in-memory columns. The JVM

ecosystem offers high-performance collections and off-heap buffers, which are natural next steps for the native kernels described in Section 6.3.

9 Conclusion

The legacy Andas reader was limited by memory amplification, not by CPU speed. Materializing the whole file, building row maps, transposing into columns, and duplicating indices made a 36 MB file unreadable on a 192 MB Android heap. Streaming columnar ingestion, a shared index, primitive storage, and algorithmic rewrites raise the readable size to 200k rows with 35 MB of retained heap on Android, and to 200k rows with 56 MB on a 256 MB JVM heap. At 50k rows, statistical and sorting kernels improve by 7–33× and hash joins by 30.9×. pandas remains faster on the desktop because of its C/NumPy engine, but the optimized library narrows the read gap and uses less memory. A 35-metric cross-language consistency check against pandas passes on both the JVM and Android with a maximum relative error of 3.17×10^{-15}. The remaining 6–20× kernel gap motivates native columnar kernels and zero-copy buffers as future work.

A Reproducibility

The repository contains the benchmark scripts, raw results, and figures. The main commands are:

```
python3 bench/gen_data.py
./bench/prepare_baseline.sh 9bb6715
python3 bench/run_benchmarks.py
python3 bench/run_pandas_benchmarks.py
python3 bench/compare_pandas.py
python3 bench/plot_results.py
```

The Android experiment is an instrumentation test:

```
export ANDROID_USER_HOME="$PWD/.android-home"
TEST=cn.ac.oac.libs.andas.CsvBenchInstrumentedTest
adb logcat -c
./gradlew :android-pandas:connectedDebugAndroidTest \
  -Pandroid.testInstrumentationRunnerArguments.class=$TEST
adb logcat -d -s ANDAS_BENCH:I
```

Raw JVM results, pandas comparison tables, and Android logcat output are stored under `bench/results/`. The LaTeX source of this report is in `report/` and can be rebuilt with `latexmk -pdf andas-technical-report.tex`.

References

[1] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.

[2] pandas Development Team, "pandas: powerful Python data analysis toolkit," `https://pandas.pydata.org/`, 2026.

[3] JetBrains, "Kotlin Programming Language," `https://kotlinlang.org/`, 2026.

[4] Google, "Android Runtime (ART) and Dalvik," `https://source.android.com/docs/core/runtime`, 2026.

[5] Apache Software Foundation, "Apache Arrow: A cross-language development platform for in-memory data," `https://arrow.apache.org/`, 2026.

[6] Oracle, "Java Microbenchmark Harness (JMH)," https://openjdk.org/projects/code-tools/jmh/, 2026.